dplyr & SQL

Lecture 16

Dr. Colin Rundel

Why databases?

Numbers every programmer should know

Historical estimates (circa 2012) illustrate the differences in scale. Actual timings depend on the hardware and workload.

Task	Time (µs)	Roughly
L1 cache reference	5×10^{-4}	0.5 ns
L2 cache reference	7×10^{-3}	7 ns
Main memory reference	1×10^{-1}	100 ns
Random seek SSD	1.5×10^{2}	150 µs
Read 1 MB sequentially from memory	2.5×10^{2}	250 µs
Read 1 MB sequentially from SSD	1×10^{3}	1 ms
Disk seek	1×10^{4}	10 ms
Read 1 MB sequentially from disk	2×10^{4}	20 ms
Round trip packet CA to Netherlands	1.5×10^{5}	150 ms

Implications for big data

Using these illustrative rates, imagine we have a 10 GB flat file and want to select the rows that meet some criterion. With no other structure this requires a sequential read of the entire data set.

File Location	Performance	Time
in memory	$10\ GB \times (250\ \mu s / 1\ MB)$	2.5 seconds
on disk (SSD)	$10\ GB \times (1\ ms / 1\ MB)$	10 seconds
on disk (HD)	$10\ GB \times (20\ ms / 1\ MB)$	200 seconds

These estimates cover reading the file once. Repeated reads add to that cost, so reducing unnecessary reads can make a large difference.

Databases can skip blocks that cannot match a filter, use indexes to locate matching rows, and choose an execution plan suited to the query. Some queries still require scanning all relevant data.

So why databases?

Implementing blocks, indexes, and query planning correctly is hard,

- the optimal strategy depends on your queries, and they change,
- data is modified (inserts, updates, deletes) so the structures must stay consistent,
- multiple users may access the data at the same time,
- failures happen and the data must remain correct and recoverable.

A database management system (DBMS) handles all of this for you, and provides a standard interface (e.g. SQL) for saying what you want rather than how to get it.

Today we treat the database as a black box and focus on how R and Python talk to it. We will use DuckDB as the engine and look at it in detail next lecture.

SQL

SQL

Structured Query Language is a special purpose language for querying and modifying tabular data.

- An ANSI standard with dialect divergence (PostgreSQL, MySQL, SQLite, DuckDB, etc.)
- Declarative, a query describes the result and the database decides how to compute it
- Maps closely, but not exactly, onto the dplyr verbs we have already been using

```sql
SELECT origin, dest, COUNT(*) AS n
FROM flights
WHERE month = 10 AND day = 21
GROUP BY origin, dest
ORDER BY n DESC
LIMIT 5
```

dplyr verbs and SQL clauses

dplyr	SQL
`select(x, y)`	`SELECT x, y`
`mutate(z = x + y)`	`SELECT *, x + y AS z`
`filter(x > 1)`	`WHERE x > 1`
`arrange(desc(x))`	`ORDER BY x DESC`
`summarize(n = n(), .by = g)`	`SELECT g, COUNT(*) AS n ... GROUP BY g`
`summarize(...) |> filter(n > 10)`	`HAVING n > 10`
`distinct(x)`	`SELECT DISTINCT x`
`head(5)`	`LIMIT 5`
`left_join(y, by = "k")`	`LEFT JOIN y ON x.k = y.k`
`mutate(m = mean(x), .by = g)`	`AVG(x) OVER (PARTITION BY g)`

Clauses appear in a fixed order (`SELECT`, `FROM`, `JOIN`, `WHERE`, `GROUP BY`, `HAVING`, `ORDER BY`, `LIMIT`) regardless of the order in which they are evaluated.

DBI

DBI

The `DBI` package provides a consistent R interface to relational databases. Each database has a backend package (`RSQLite`, `RPostgres`, `duckdb`, `bigrquery`, ...) that implements the same set of generics for

- connecting and disconnecting,
- listing tables and fields,
- sending queries and fetching results,
- executing statements that modify the database,
- transactions.

Code written against `DBI` is largely portable between backends. The SQL you send is not (dialects differences).

Connecting

A connection object is created with `dbConnect()` and a driver from the backend package - the connection may be local or remote.

For DuckDB is local, so the driver takes the path of a database file. If this file/database does not exist it will be created. If no path is supplied the database is created in memory and is *discarded* when the connection closes.

```r
(con = DBI::dbConnect(duckdb::duckdb(dbdir = "data/flights.duckdb")))
```

```
<duckdb_connection 58230 driver=<duckdb_driver dbdir='/Users/rundel/Desktop/Sta523-Fa26/website/static/slides/data/flights.duckdb' read_only=FALSE bigint=numeric>>
```

Connection objects

The connection is a handle to the open database, it is what every other `DBI` function takes as its first argument.

The object itself is tiny, since the data stays in the database. Its class comes from the backend, which is how `DBI`'s generics dispatch to the right implementation.

```r
class(con)
```

```
[1] "duckdb_connection"
attr(,"package")
[1] "duckdb"
```

```r
DBI::dbIsValid(con)
```

```
[1] TRUE
```

Connections should be closed with `dbDisconnect()` when you are done. R will close a connection when the object is garbage collected, but relying on this is a bad habit.

```r
DBI::dbDisconnect(con)
```

Tables

A data frame can be written as a new table in the database with `dbWriteTable()`.

```r
employees = tibble(
  name   = c("Alice", "Bob", "Carol", "Dave"),
  salary = c(52000, 40000, 30000, 33000),
  dept   = c("Accounting", "Accounting", "Sales", "Accounting")
)
```

```r
DBI::dbWriteTable(con, "employees", employees)
```

```r
DBI::dbListTables(con)
```

```
[1] "employees"
```

```r
DBI::dbListFields(con, "employees")
```

```
[1] "name"   "salary" "dept"
```

```r
DBI::dbExistsTable(con, "sales")
```

```
[1] FALSE
```

Queries

`dbGetQuery()` sends a query and collects a data frame as the result.

```r
DBI::dbGetQuery(con, "SELECT dept, AVG(salary) AS avg FROM employees GROUP BY dept")
```

```
        dept      avg
1 Accounting 41666.67
2      Sales 30000.00
```

It wraps the three step process of sending the query, fetching rows, and clearing the result. The steps can be run separately when a result is too large (or slow) to fetch at once.

```r
res = DBI::dbSendQuery(
  con, "SELECT * FROM employees"
)
DBI::dbFetch(res, n = 2)
```

```
   name salary       dept
1 Alice  52000 Accounting
2   Bob  40000 Accounting
```

```r
DBI::dbHasCompleted(res)
```

```
[1] FALSE
```

```r
DBI::dbFetch(res)
```

```
   name salary       dept
1 Carol  30000      Sales
2  Dave  33000 Accounting
```

```r
DBI::dbClearResult(res)
```

Modifying

Statements that change the database rather than return rows (`INSERT`, `UPDATE`, `DELETE`, `CREATE`, `DROP`) are run with `dbExecute()`, which returns the number of rows affected.

```R
DBI::dbExecute(con, "INSERT INTO employees VALUES ('Eve', 44000, 'Sales')")
```

```
[1] 1
```

```R
DBI::dbExecute(con, "UPDATE employees SET salary = salary * 1.1 WHERE dept = 'Sales'")
```

```
[1] 2
```

`DBI` also has helpers for common changes: `dbAppendTable()` adds the rows of a data frame and `dbRemoveTable()` drops a table.

```R
DBI::dbRemoveTable(con, "employees")
DBI::dbListTables(con)
```

```
character(0)
```

Parameterized queries

Never paste user supplied values into a query string. Quoting is easy to get wrong and a hostile value can change the meaning of the query (SQL injection). Instead leave `?` placeholders and pass the values separately.

```r
one = "SELECT * FROM airlines WHERE carrier = ?"
two = "SELECT * FROM airlines WHERE carrier IN (?, ?)"
```

```r
DBI::dbGetQuery(
  con, one, params = list("UA")
)
```

```
  carrier                name
1      UA United Air Lines Inc.
```

```r
DBI::dbGetQuery(
  con, two, params = list("UA", "DL")
)
```

```
  carrier                 name
1      DL  Delta Air Lines Inc.
2      UA United Air Lines Inc.
```

For values that must be interpolated into the string itself, such as table or column names, use

Python's DB-API

Python's equivalent to `DBI` is the DB-API specification (PEP 249). Each database has a package (`sqlite3`, `psycopg`, `duckdb`, …) that implements the specification. A connection creates cursors, and a cursor runs a query and fetches rows as tuples.

```py
import duckdb
con = duckdb.connect()
query = "SELECT origin, COUNT(*) AS n FROM 'data/flights.parquet' GROUP BY origin"
```

```py
cur = con.cursor()
cur.execute(query)
```

```
<_duckdb.DuckDBPyConnection object at 0x116deaf20>
```

```py
[d[0] for d in cur.description]
```

```
['origin', 'n']
```

```py
cur.fetchone()
```

```
('LGA', 104662)
```

```py
cur.fetchall()
```

```
[('JFK', 111279), ('EWR', 120835)]
```

```py
pl.read_database(query, con)
```

shape: (3, 2)

origin	n
str	i64
"LGA"	104662
"JFK"	111279
"EWR"	120835

`pl.read_database()` accepts any DB-API connection and returns a frame. Querying a Parquet file as a table is a DuckDB

dbplyr

Database tables in dplyr

The `dbplyr` package lets dplyr work with tables in a database. It is installed with the tidyverse and loaded automatically when needed.

It allows dplyr verbs to operate on a reference to a table in the database. `tbl()` points at an existing table, `copy_to()` writes a local data frame to the database and returns a reference to it. The data stays in the database, the R object holds only the connection and the query that produces the table.

```r
flights_tbl = copy_to(con, nycflights13::flights, "flights", temporary = FALSE)
planes_tbl = copy_to(con, nycflights13::planes, "planes", temporary = FALSE)
airlines_tbl = tbl(con, "airlines")
```

```r
class(flights_tbl)
```

```
[1] "tbl_duckdb_connection" "tbl_dbi"               "tbl_sql"
[4] "tbl_lazy"              "tbl"
```

```r
lobstr::obj_size(flights_tbl)
```

```
42.89 kB
```

```r
lobstr::obj_size(nycflights13::flights)
```

```
40.65 MB
```

Verbs work as before

The dplyr verbs are all available and the results print like a df tibble.

```r
(oct_21 = flights_tbl |>
  filter(month == 10, day == 21) |>
  select(origin, dest, tailnum, dep_delay)
)
```

```
# A query:  ?? x 4
# Database: DuckDB [flights.duckdb]
  origin dest  tailnum dep_delay
  <chr>  <chr> <chr>       <dbl>
1 EWR    CLT   N152UW         -7
2 EWR    IAH   N535UA         -7
3 JFK    MIA   N5BSAA         -6
4 JFK    SJU   N531JB         -6
5 JFK    BQN   N827JB         -8
6 LGA    IAH   N15710         -2
# ℹ more rows
```

```r
collect(oct_21)
```

```
# A tibble: 991 × 4
  origin dest  tailnum dep_delay
  <chr>  <chr> <chr>       <dbl>
1 EWR    CLT   N152UW         -7
2 EWR    IAH   N535UA         -7
3 JFK    MIA   N5BSAA         -6
4 JFK    SJU   N531JB         -6
5 JFK    BQN   N827JB         -8
6 LGA    IAH   N15710         -2
# ℹ 985 more rows
```

Laziness

A database table is lazy. Each verb builds a description of the query. Printing executes the query but fetches only enough rows for a preview.

```r
nrow(oct_21)
```

```
[1] NA
```

```r
oct_21 |> count()
```

```
# A query:  ?? x 1
# Database: DuckDB [flights.duckdb]
      n
  <dbl>
1   991
```

Building a pipeline lets the database combine and optimize the operations.

We use `collect()` to bring the full result into R as a tibble, or `pull()` to retrieve a single column.

show_query()

`show_query()` can be used to print the query that will be sent.

```r
oct_21 |>
  show_query()
```

```
<SQL>
SELECT origin, dest, tailnum, dep_delay
FROM flights
WHERE ("month" = 10.0) AND ("day" = 21.0)
```

```r
oct_21 |>
  summarize(
    delay = mean(dep_delay, na.rm = TRUE),
    .by = origin
  ) |>
  show_query()
```

```
<SQL>
SELECT origin, AVG(dep_delay) AS delay
FROM flights
WHERE ("month" = 10.0) AND ("day" = 21.0)
GROUP BY origin
```

Joins

Join verbs work between tables in the same database and translate to SQL `JOIN` clauses.

```r
flights_tbl |>
  left_join(airlines_tbl, by = "carrier") |>
  count(name, sort = TRUE)
```

```
# A query:    ?? x 2
# Database:   DuckDB [flights.duckdb]
# Ordered by: desc(n)
  name                        n
  <chr>                   <dbl>
1 United Air Lines Inc.   58665
2 JetBlue Airways         54635
3 ExpressJet Airlines Inc. 54173
4 Delta Air Lines Inc.    48110
5 American Airlines Inc.  32729
6 Envoy Air               26397
# ℹ more rows
```

```r
flights_tbl |>
  left_join(airlines_tbl, by = "carrier") |>
  count(name, sort = TRUE) |>
  show_query()
```

```
<SQL>
SELECT "name", COUNT(*) AS n
FROM (
  SELECT flights.*, "name"
  FROM flights
  LEFT JOIN airlines
    ON (flights.carrier = airlines.carrier)
) AS q01
GROUP BY "name"
ORDER BY n DESC
```

Window functions

A grouped `mutate()` or `filter()` needs a per group value on every row. SQL does this with window functions, an aggregate followed by an `OVER (PARTITION BY ...)` clause.

```r
flights_tbl |>
  mutate(n = n(), .by = dest) |>
  select(
    origin, dest, carrier, tailnum, arr_delay, n
  )
```

```
# A query:  ?? x 6
# Database: DuckDB [flights.duckdb]
  origin dest  carrier tailnum arr_delay     n
  <chr>  <chr> <chr>   <chr>       <dbl> <dbl>
1 EWR    IAH   UA      N14228         11  7198
2 LGA    IAH   UA      N24211         20  7198
3 JFK    MIA   AA      N619AA         33 11728
4 JFK    BQN   B6      N804JB        -18   896
5 LGA    ATL   DL      N668DN        -25 17215
6 EWR    ORD   UA      N39463         12 17283
# ℹ more rows
```

```r
flights_tbl |>
  mutate(n = n(), .by = dest) |>
  select(
    origin, dest, carrier, tailnum, arr_delay, n
  ) |>
  show_query()
```

```
<SQL>
SELECT
  origin,
  dest,
  carrier,
  tailnum,
  arr_delay,
  COUNT(*) OVER (PARTITION BY dest) AS n
FROM flights
```

`collect()`, `compute()`, `collapse()`

These functions control how a query is executed or composed.

- `collect()` runs the query and returns the result as a tibble in R,
- `compute()` runs the query and stores the result in a temporary table in the database, returning a reference to it. Use it when an expensive intermediate result is reused by several later queries,
- `collapse()` runs nothing but wraps the query so far in a subquery, which is occasionally needed to control how later verbs are translated.

```r
by_origin = flights_tbl |>
  summarize(n = n(), .by = origin) |>
  compute()
show_query(by_origin)
```

```
<SQL>
SELECT *
FROM dbplyr_tmp_e8RAZNJBvn
```

Mixing local data

Because the work happens in the database, a database table cannot be combined directly with a local data frame.

```r
big_hubs = tibble(dest = c("ORD", "ATL", "LAX"))
```

```r
flights_tbl |> semi_join(big_hubs, by = "dest")
```

```
Error in `semi_join()`:
! `x` and `y` must share the same source.
ℹ Use `copy = "temp-table"` to copy `y` to a temporary table.
ℹ USe `copy = "inline"` to convert `y` to inline SQL.
```

The join verbs can copy the local table into the database for you with `copy = TRUE`, or use `copy_to()` to do it yourself first.

```r
flights_tbl |> semi_join(big_hubs, by = "dest", copy = TRUE) |> count(dest)
```

```
# A query:  ?? x 2
# Database: DuckDB [flights.duckdb]
  dest      n
  <chr> <dbl>
1 ATL   17215
2 ORD   17283
3 LAX   16174
```

Translation

dbplyr translates many common vector functions from base R, `stringr`, and `lubridate` into SQL. Support depends on the database backend.

`translate_sql()` lets you see the translation of an expression on its own.

```R
dbplyr::translate_sql(x %in% c("a", "b") & y > 10, con = con)
```

```
<SQL> x IN ('a', 'b') AND y > 10.0
```

```R
dbplyr::translate_sql(if_else(x > 0, "late", "on time"), con = con)
```

```
<SQL> CASE WHEN (x > 0.0) THEN 'late' WHEN NOT (x > 0.0) THEN 'on time' END
```

```R
dbplyr::translate_sql(stringr::str_detect(x, "^N1"), con = con)
```

```
<SQL> REGEXP_MATCHES(x, '^N1')
```

```R
dbplyr::translate_sql(lubridate::year(x), con = con)
```

```
<SQL> EXTRACT(year FROM x)
```

Dialects

The connection matters since the translation is specific to the engine. `dbplyr` ships `simulate_*()` connections for checking a translation without connecting to an actual database.

```R
dbplyr::translate_sql(paste(x, y), con = con)
```

```
<SQL> CONCAT_WS(' ', x, y)
```

```R
dbplyr::translate_sql(paste(x, y), con = dbplyr::simulate_sqlite())
```

```
<SQL> `x` || ' ' || `y`
```

```R
dbplyr::translate_sql(paste(x, y), con = dbplyr::simulate_mssql())
```

```
<SQL> [x] + ' ' + [y]
```

Unknown functions

When dbplyr meets a function it does not know, it passes the call through unchanged, and leaves it to the database to decide whether it makes sense.

```r
flights_tbl |> mutate(z = foo(dep_delay)) |> show_query()
```

```
<SQL>
SELECT *, foo(dep_delay) AS z
FROM flights
```

Running this query is an error, since DuckDB has no `foo()`. The flip side is that any function the database does have can be called from dplyr without a translation, like DuckDB's `regexp_extract()`.

```r
flights_tbl |>
  mutate(n_number = regexp_extract(tailnum, "^N(\\d+)", 1L)) |>
  select(tailnum, n_number) |>
  head(3)
```

```
# A query:  ?? x 2
# Database: DuckDB [flights.duckdb]
  tailnum n_number
  <chr>   <chr>
1 N14228  14228
2 N24211  24211
3 N619AA  619
```

Differences from data frames

Database tables have no intrinsic row order, so `slice()`, `slice_head()`, and `slice_tail()` are unavailable. `head()`, `slice_min()`, `slice_max()`, and `slice_sample()` all work, and `arrange()` before `head()` gives an ordered result.

Translation can also change behavior - SQL aggregates such as `AVG()` and `SUM()` ignore `NULL` values, so dbplyr warns when translating `mean()` without `na.rm = TRUE`. Supplying it acknowledges that behavior and silences the warning.

```r
flights_tbl |> summarize(delay = mean(dep_delay)) |> show_query()
```

```
Warning: Missing values are always removed in SQL aggregation functions.
Use `na.rm = TRUE` to silence this warning
This warning is displayed once every 8 hours.

<SQL>
SELECT AVG(dep_delay) AS delay
FROM flights
```

Exercise 1

Using `flights_tbl` and `planes_tbl`, answer the following with dplyr. For each, look at the result of `show_query()` and identify which verbs produced which clauses.

1. For each origin airport, which destination received the most flights and how many were there?
2. Which five manufacturers' planes flew the most flights? Report the number of flights and the mean departure delay for each.
3. Repeat question 2 without the five row limit, keeping only manufacturers with at least 30,000 flights. Where does this condition appear in the SQL?

polars & SQL

pl.sql()

`polars` has a SQL interface that compiles a query into the same lazy plan as the expression API. `pl.sql()` looks up each table name among the frames in scope and returns a `LazyFrame`.

```py
flights = pl.scan_parquet("data/flights.parquet")
planes = pl.scan_parquet("data/planes.parquet")
```

```py
pl.sql("""
  SELECT origin, dest, COUNT(*) AS n
  FROM flights
  WHERE month = 10 AND day = 21
  GROUP BY origin, dest
  ORDER BY n DESC LIMIT 3
""").collect()
```

shape: (3, 3)

origin	dest	n
str	str	i64
"JFK"	"LAX"	32
"LGA"	"ORD"	31
"LGA"	"ATL"	30

```py
pl.sql("""
  SELECT p.manufacturer, COUNT(*) AS n
  FROM flights f
  JOIN planes p ON f.tailnum = p.tailnum
  GROUP BY p.manufacturer
  ORDER BY n DESC LIMIT 3
""", eager=True)
```

shape: (3, 2)

manufacturer	n
str	i64
"BOEING"	82912
"EMBRAER"	66068
"AIRBUS"	47302

One plan, two languages

Polars parses SQL into expressions for its own execution engine. In this example, SQL and the equivalent method chain produce the same plan apart from an integer cast. This reverses the direction of dbplyr, which turns dplyr into SQL for a database to run.

```py
q = pl.sql("SELECT origin, COUNT(*) AS n FROM flights GROUP BY origin")
print(q.explain())
```

```
AGGREGATE[maintain_order: false]
  [len().cast(Int64).alias("n")] BY [col("origin")]
  FROM
  Parquet SCAN [data/flights.parquet]
  PROJECT 1/19 COLUMNS
  ESTIMATED ROWS: 336776
```

```py
q = flights.group_by("origin").agg(n = pl.len())
print(q.explain())
```

```
AGGREGATE[maintain_order: false]
  [len().alias("n")] BY [col("origin")]
  FROM
  Parquet SCAN [data/flights.parquet]
  PROJECT 1/19 COLUMNS
  ESTIMATED ROWS: 336776
```

What is supported

The `polars` SQL dialect covers the core of a `SELECT` query (`WHERE`, `GROUP BY`, `HAVING`, `ORDER BY`, `LIMIT`, joins, subqueries, `CASE`, common table expressions (`WITH`), window functions) and a few hundred functions for strings, dates, and regular expressions.

```py
pl.sql("""
  SELECT tailnum, tailnum ~ '^N1' AS n1
  FROM flights LIMIT 3
""").collect()
```

shape: (3, 2)

tailnum	n1
str	bool
"N14228"	true
"N24211"	false
"N619AA"	false

```py
pl.sql("""
  SELECT foo(dep_delay) FROM flights
""").collect()
```

```
polars.exceptions.SQLInterfaceError:
unsupported function 'foo'
```

Unknown functions are an error because there is no database behind the query to interpret them. Polars SQL also supports operations such as `DELETE`, which returns the remaining rows without modifying the original frame.

Summary

Takeaways

- A database combines storage structures and a query planner to execute queries efficiently and avoid unnecessary reads. SQL is the standard language for asking it questions.
- `DBI` is the R interface to any relational database. Use `dbGetQuery()` for queries, `dbExecute()` for statements that modify the database, and `?` placeholders for values rather than pasting them into the query.
- `dbplyr` lets dplyr verbs run against a database table. Queries are lazy and translated to SQL, `show_query()` reveals the translation, and `collect()` brings the result into R.
- dbplyr translates many common vector functions from base R, `stringr`, and `lubridate` into SQL. Support depends on the database backend. Unknown functions are passed through to the database, which makes any database function usable but means mistakes surface only when the query runs.
- `polars` goes the other direction, compiling SQL into the same lazy plan as its expression API, so SQL and methods can be mixed freely in a single pipeline.

Learning more

- R4DS - Databases
- dbplyr - Introduction and Function translation
- DBI - Getting started
- polars - SQL interface
- SQL for Data Scientists by Renée Teate